

Задачі кластеризації та класифікації

Мета

Ознайомитись з різновидами моделей для задач кластеризації та класифікації, а також методами побудови та оцінки цих моделей. Після завершення цієї лабораторної роботи ви зможете:

- Використовувати прості моделі для задач кластеризації та класифікації
- Використовувати перехресну перевірку для оцінки якості моделі
- Обирати оптимальну складність моделі для уникнення перенавчання
- Вдосконалювати моделі за допомогою підбору параметрів

Завдання, що оцінюються

1. Скачайте дані з фінального файлу першої роботи з виправленими помилками та заповненими пропусками. Виконайте кластеризацію по ВВП на душу населення та щільності населення.
2. Використайте метод ліктя для підбору оптимальної кількості кластерів.
3. Визначіть, який регіон домінує в кожному з кластерів.
4. Побудуйте кілька (3-5) моделей класифікації, що визначають регіон, до якого належить країна, по ознаках 'GDP per capita', 'Population', 'CO2 emission', 'Area'. Оцініть точність класифікації (використайте 20% загального набору в якості тестових даних).
5. Для однієї з моделей виконайте підбір параметра. Обґрунтуйте ваш вибір.

Завдання #1:

Виконайте кластеризацію по ВВП на душу населення та щільності населення.

Зчитую дані з файлу у датафрейм

```
In [29]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.ensemble import ExtraTreesClassifier, RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, silhouette_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv('data/clean_data.csv')
df.head()
```

```
Out [29]:
```

	Country Name	Region	GDP per capita	Population	CO2 emission	Area	Density
0	Afghanistan	South Asia	561.778746	34656032.0	9809.225	652860.0	53.083405
1	Albania	Europe & Central Asia	4124.982390	2876101.0	5716.853	28750.0	100.038296
2	Algeria	Middle East & North Africa	3916.881571	40606052.0	145400.217	2381740.0	17.048902
3	American Samoa	East Asia & Pacific	11834.745230	55599.0	6501.591	200.0	277.995000
4	Andorra	Europe & Central Asia	36988.622030	77281.0	462.042	470.0	164.427660

Виділяю параметри для кластеризації

```
In [30]: cluster_features = df[['GDP per capita', 'Density']]
cluster_scaler = StandardScaler()
cluster_features_scaled = cluster_scaler.fit_transform(cluster_features)

cluster_features.describe()
```

Out [30]:

	GDP per capita	Density
count	217.000000	217.000000
mean	12997.297937	429.949541
std	17212.901697	2012.114136
min	285.727442	0.006594
25%	2175.673442	35.184097
50%	6049.232578	91.797365
75%	14900.778900	221.344014
max	100738.684200	20203.531353

Будую модель методом k середніх з кількістю кластерів 8

In [31]:

```
kmeans8 = KMeans(  
    init='k-means++',  
    n_clusters=8,  
    n_init=20,  
    max_iter=300,  
    random_state=42,  
)  
  
kmeans8.fit(cluster_features_scaled)
```

Out [31]:

▼ KMeans ⓘ ?

► Parameters

► Fitted attributes

► 8 features 📄

Отримані центри кластерів:

In [32]:

```
centers8 = pd.DataFrame(  
    cluster_scaler.inverse_transform(kmeans8.cluster_centers_),  
    columns=cluster_features.columns,  
)  
centers8.index.name = 'Cluster'  
centers8.round(2)
```

Out [32]:

	GDP per capita	Density
Cluster		
0	3689.12	140.88
1	14900.78	19249.50
2	36237.83	207.78
3	83831.44	147.14
4	48351.74	7223.66
5	15595.82	351.10
6	54877.86	115.28
7	74017.18	20203.53

Масив із номерами кластерів для кожного рядка даних:

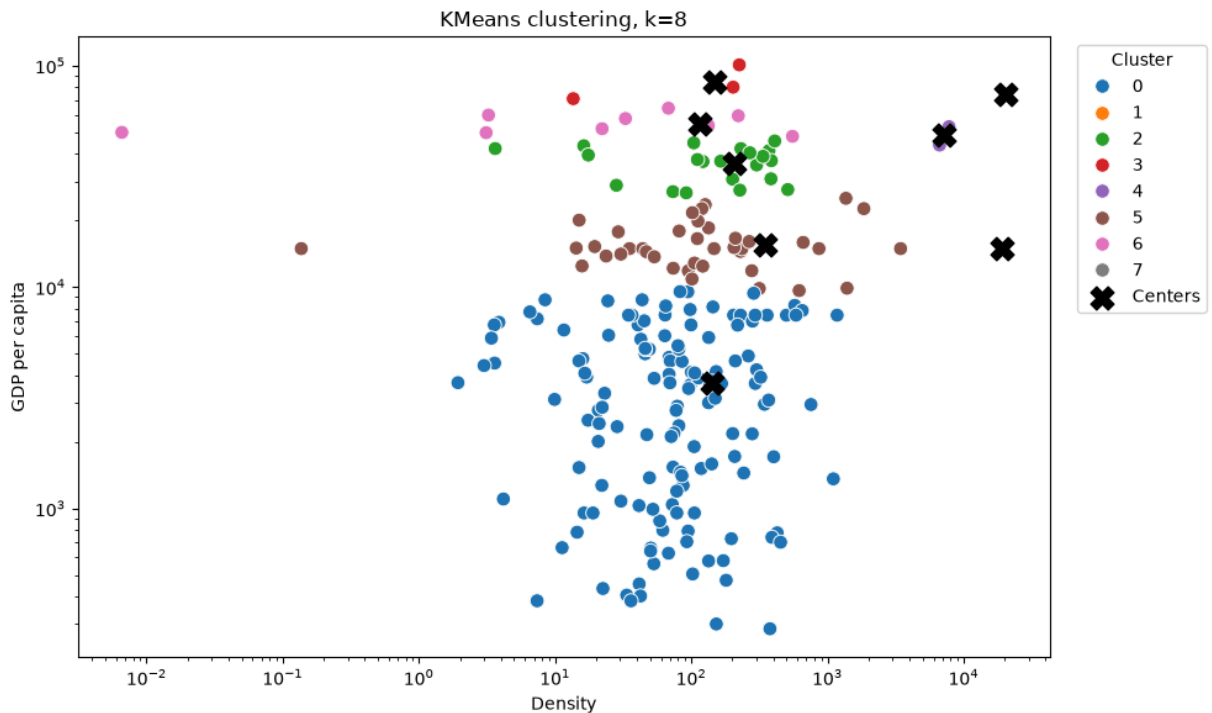
In [33]: kmeans8.labels_

Out [33]: array([0, 0, 0, 5, 2, 0, 5, 5, 0, 0, 6, 2, 0, 2, 5, 0, 5, 0, 2, 0, 0, 6,
0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 2, 0, 0, 0, 5, 5, 0, 0, 0,
0, 0, 5, 0, 5, 0, 0, 5, 5, 6, 0, 0, 0, 0, 0, 0, 0, 0, 5, 0, 5, 0,
2, 2, 0, 0, 0, 0, 2, 0, 5, 5, 5, 5, 2, 0, 0, 0, 0, 0, 0, 4, 5, 6,
0, 0, 0, 0, 6, 5, 2, 2, 0, 2, 0, 0, 0, 0, 0, 2, 0, 2, 0, 0, 5, 0,
0, 0, 0, 5, 5, 3, 7, 0, 0, 0, 0, 5, 0, 5, 0, 0, 5, 0, 0, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0, 2, 0, 2, 0, 0, 0, 5, 3, 5, 0, 5, 5, 0, 0, 0,
0, 5, 5, 2, 6, 0, 0, 0, 0, 6, 0, 5, 0, 0, 5, 0, 4, 0, 5, 5, 0, 0,
0, 0, 2, 0, 5, 0, 0, 0, 0, 0, 0, 6, 3, 0, 0, 0, 0, 0, 0, 0, 5, 0,
5, 0, 0, 0, 0, 0, 2, 2, 6, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
dtype=int32)

Візуалізую отримані кластери:

```
In [34]: plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x='Density',
    y='GDP per capita',
    hue=kmeans8.labels_,
    palette='tab10',
    s=70,
)
plt.scatter(
    centers8['Density'],
    centers8['GDP per capita'],
    c='black',
    marker='X',
    s=180,
    label='Centers',
)
plt.xscale('log')
plt.yscale('log')
```

```
plt.xlabel('Density')
plt.ylabel('GDP per capita')
plt.title('KMeans clustering, k=8')
plt.legend(title='Cluster', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



Завдання #2:

Використайте метод ліктя для підбору оптимальної кількості кластерів.

Визначаю оптимальну кількість кластерів. Скористаюсь методом "ліктя". Для цього ініціалізую алгоритм k середніх кількістю кластерів від 1 до 10 і для кожної моделі рахую суму квадратів похибок (евклідових відстаней точок кластерів від відповідних центрів):

```
In [35]: kmeans_kwargs = {
    'init': 'k-means++',
    'n_init': 20,
    'max_iter': 300,
    'random_state': 42,
}

sse = []
silhouette_coefficients = []
max_clusters = 10
```

```

for k in range(1, max_clusters + 1):
    kmeans = KMeans(n_clusters=k, **kmeans_kwargs)
    kmeans.fit(cluster_features_scaled)
    sse.append(kmeans.inertia_)
    if k > 1:
        silhouette_coefficients.append(silhouette_score(cluster_features_scaled, kmeans.labels_))

elbow_results = pd.DataFrame({
    'k': range(1, max_clusters + 1),
    'SSE': sse,
    'Silhouette': [np.nan] + silhouette_coefficients,
})
elbow_results.round(4)

```

Out [35]:

	k	SSE	Silhouette
0	1	434.0000	NaN
1	2	240.7107	0.8977
2	3	84.2926	0.7520
3	4	61.0172	0.7601
4	5	40.1598	0.6338
5	6	26.8736	0.6299
6	7	20.8357	0.6257
7	8	16.2085	0.6036
8	9	12.8738	0.5670
9	10	10.1557	0.5287

Візуалізую отримані результати:

```

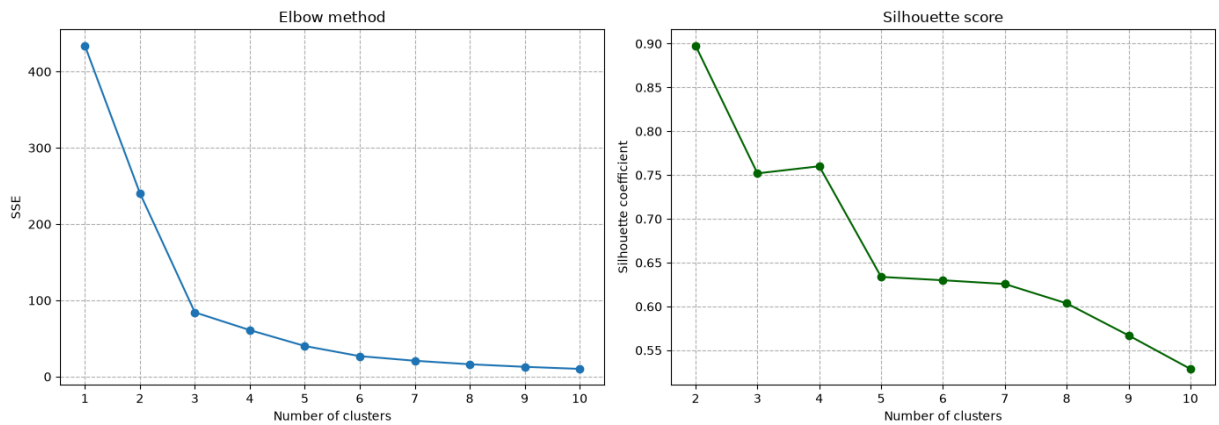
In [36]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(range(1, max_clusters + 1), sse, marker='o')
axes[0].set_xticks(range(1, max_clusters + 1))
axes[0].set_xlabel('Number of clusters')
axes[0].set_ylabel('SSE')
axes[0].set_title('Elbow method')
axes[0].grid(linestyle='--')

axes[1].plot(range(2, max_clusters + 1), silhouette_coefficients, marker='o')
axes[1].set_xticks(range(2, max_clusters + 1))
axes[1].set_xlabel('Number of clusters')
axes[1].set_ylabel('Silhouette coefficient')
axes[1].set_title('Silhouette score')
axes[1].grid(linestyle='--')

plt.tight_layout()
plt.show()

```



Оптимальна кількість кластерів за методом ліктя дорівнює **3**: після цього значення SSE зменшується вже значно повільніше.

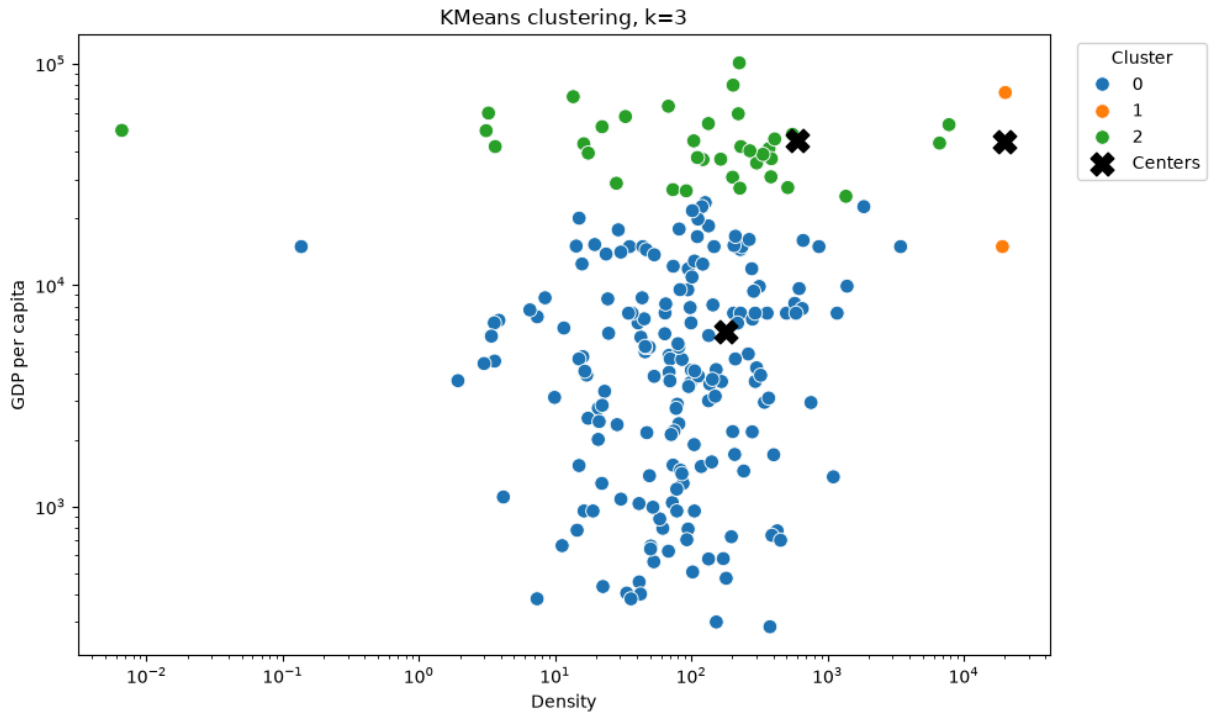
Візуалізую для оптимальної кількості кластерів:

```
In [37]: optimal_k = 3
optimal_kmeans = KMeans(n_clusters=optimal_k, **kmeans_kwargs)
optimal_kmeans.fit(cluster_features_scaled)

optimal_centers = pd.DataFrame(
    cluster_scaler.inverse_transform(optimal_kmeans.cluster_centers_),
    columns=cluster_features.columns,
)
optimal_centers.index.name = 'Cluster'

plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x='Density',
    y='GDP per capita',
    hue=optimal_kmeans.labels_,
    palette='tab10',
    s=70,
)
plt.scatter(
    optimal_centers['Density'],
    optimal_centers['GDP per capita'],
    c='black',
    marker='X',
    s=180,
    label='Centers',
)
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Density')
plt.ylabel('GDP per capita')
plt.title('KMeans clustering, k=3')
plt.legend(title='Cluster', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

```
optimal_centers.round(2)
```



Out [37]:

	GDP per capita	Density
Cluster		
0	6163.45	179.84
1	44458.98	19726.52
2	45228.84	601.53

Завдання #3:

Визначіть, який регіон домінує в кожному з кластерів.

Додаю мітки кластерів в датафрейм

```
In [38]: df_clustered = df.copy()
df_clustered['Cluster'] = optimal_kmeans.labels_
df_clustered.head()
```

Out [38]:

	Country Name	Region	GDP per capita	Population	CO2 emission	Area	Density
0	Afghanistan	South Asia	561.778746	34656032.0	9809.225	652860.0	53.083405
1	Albania	Europe & Central Asia	4124.982390	2876101.0	5716.853	28750.0	100.038296
2	Algeria	Middle East & North Africa	3916.881571	40606052.0	145400.217	2381740.0	17.048902
3	American Samoa	East Asia & Pacific	11834.745230	55599.0	6501.591	200.0	277.995000
4	Andorra	Europe & Central Asia	36988.622030	77281.0	462.042	470.0	164.427660

Визначаю домінуючий регіон для кожного кластера

```
In [39]: cluster_region_counts = pd.crosstab(df_clustered['Cluster'], df_clustered['Region'])
dominant_region_summary = pd.DataFrame({
    'Dominant region': cluster_region_counts.idxmax(axis=1),
    'Countries in dominant region': cluster_region_counts.max(axis=1),
    'Cluster size': cluster_region_counts.sum(axis=1),
})

display(cluster_region_counts)
dominant_region_summary
```

Region	East Asia & Pacific	Europe & Central Asia	Latin America & Caribbean	Middle East & North Africa	North America	South Asia	Sub-Saharan Africa
Cluster							
0	28	39	40	16	0	8	48
1	1	1	0	0	0	0	0
2	8	18	2	5	3	0	0

Out [39]:

	Dominant region	Countries in dominant region	Cluster size
Cluster			
0	Sub-Saharan Africa	48	179
1	East Asia & Pacific	1	2
2	Europe & Central Asia	18	36

Завдання #4:

Побудуйте кілька (3-5) моделей класифікації, що визначають регіон, до якого належить країна, по ознаках 'GDP per capita', 'Population', 'CO2 emission', 'Area'. Оцініть точність класифікації (використайте 20% загального набору в якості тестових даних).

Обираю потрібні ознаки:

```
In [40]: classification_features = ['GDP per capita', 'Population', 'CO2 emission', 'Area']
         target = 'Region'

X = df[classification_features]
y = df[target]

X.head()
```

```
Out [40]:
```

	GDP per capita	Population	CO2 emission	Area
0	561.778746	34656032.0	9809.225	652860.0
1	4124.982390	2876101.0	5716.853	28750.0
2	3916.881571	40606052.0	145400.217	2381740.0
3	11834.745230	55599.0	6501.591	200.0
4	36988.622030	77281.0	462.042	470.0

Розділяю датасет на навчальну і тестову вибірки за допомогою функції `train_test_split()`:

```
In [41]: df_train, df_test = train_test_split(
         df,
         test_size=0.2,
         random_state=42,
         stratify=df[target],
         )
```

```
df_train.shape, df_test.shape
```

```
Out[41]: ((173, 7), (44, 7))
```

Розміщую цільові дані - Region - в окремому датафреймі:

```
In [42]: x_train = df_train[classification_features]
y_train = df_train[target]

x_test = df_test[classification_features]
y_test = df_test[target]
```

Навчання та тестування моделей:

Для навчання були обрані наступні методи:

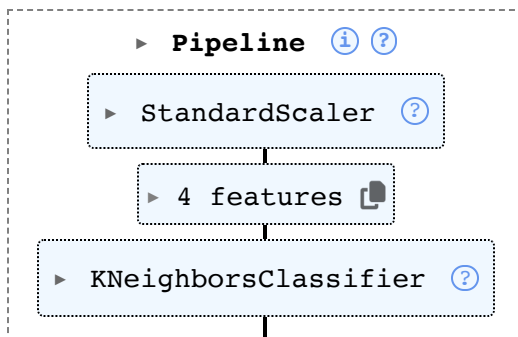
- k-nearest neighbors;
- support vector machine;
- decision tree;
- random forest;
- extra trees.

Метод 1

Будую модель

```
In [43]: KNN_model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=7),
)
KNN_model.fit(x_train, y_train)
```

```
Out[43]:
```



Оцінюю точність

```
In [44]: classification_results = {}

def evaluate_classifier(name, model):
    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)
```

```

classification_results[name] = accuracy
print(f'{name} accuracy = {accuracy:.4f}')
print(classification_report(y_test, y_pred, zero_division=0))
return y_pred

```

```
knn_pred = evaluate_classifier('KNN', KNN_model)
```

KNN accuracy = 0.4545

	precision	recall	f1-score	support
East Asia & Pacific	0.30	0.43	0.35	7
Europe & Central Asia	0.47	0.58	0.52	12
Latin America & Caribbean	0.25	0.25	0.25	8
Middle East & North Africa	0.00	0.00	0.00	4
North America	0.00	0.00	0.00	1
South Asia	0.00	0.00	0.00	2
Sub-Saharan Africa	0.80	0.80	0.80	10
accuracy			0.45	44
macro avg	0.26	0.29	0.27	44
weighted avg	0.40	0.45	0.42	44

Support Vector Machine

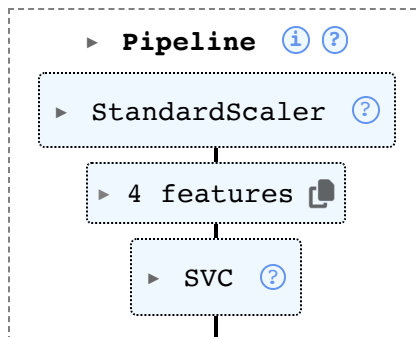
Будую модель

```

In [45]: SVC_model = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=10, gamma='scale', random_state=42),
)
SVC_model.fit(x_train, y_train)

```

Out [45]:



Оцінюю точність

```
In [46]: svc_pred = evaluate_classifier('SVC', SVC_model)
```

SVC accuracy = 0.4545

	precision	recall	f1-score	support
East Asia & Pacific	0.00	0.00	0.00	7
Europe & Central Asia	0.56	0.75	0.64	12
Latin America & Caribbean	0.22	0.25	0.24	8
Middle East & North Africa	0.00	0.00	0.00	4
North America	1.00	1.00	1.00	1
South Asia	0.00	0.00	0.00	2
Sub-Saharan Africa	0.57	0.80	0.67	10
accuracy			0.45	44
macro avg	0.34	0.40	0.36	44
weighted avg	0.35	0.45	0.39	44

Decision Tree

Будую модель

```
In [47]: decision_tree_model = DecisionTreeClassifier(max_depth=5, random_state=42)
decision_tree_model.fit(x_train, y_train)
```

Out[47]:

▼ DecisionTreeClassifier ⓘ ?

- Parameters
- Fitted attributes

Оцінюю точність

```
In [48]: decision_tree_pred = evaluate_classifier('Decision Tree', decision_tree_model)
```

Decision Tree accuracy = 0.4773

	precision	recall	f1-score	support
East Asia & Pacific	0.23	0.43	0.30	7
Europe & Central Asia	0.50	0.50	0.50	12
Latin America & Caribbean	0.43	0.38	0.40	8
Middle East & North Africa	0.25	0.25	0.25	4
North America	0.00	0.00	0.00	1
South Asia	1.00	0.50	0.67	2
Sub-Saharan Africa	1.00	0.70	0.82	10
accuracy			0.48	44
macro avg	0.49	0.39	0.42	44
weighted avg	0.55	0.48	0.50	44

Random Forest

Будую модель

```
In [49]: random_forest_model = RandomForestClassifier(
          n_estimators=200,
          max_depth=8,
          random_state=42,
          class_weight='balanced',
        )
random_forest_model.fit(x_train, y_train)
```

```
Out[49]: ▼ RandomForestClassifier ⓘ ?
          ► Parameters
          ► Fitted attributes
```

Оцінюю точність

```
In [50]: random_forest_pred = evaluate_classifier('Random Forest', random_forest_model)
```

```
Random Forest accuracy = 0.5227
```

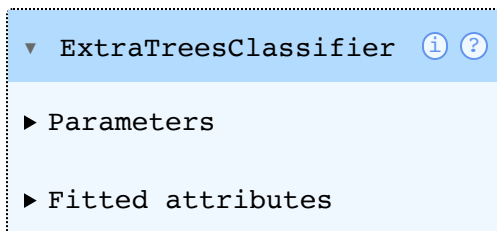
	precision	recall	f1-score	support
East Asia & Pacific	0.38	0.43	0.40	7
Europe & Central Asia	0.69	0.75	0.72	12
Latin America & Caribbean	0.25	0.25	0.25	8
Middle East & North Africa	0.33	0.25	0.29	4
North America	1.00	1.00	1.00	1
South Asia	0.25	0.50	0.33	2
Sub-Saharan Africa	0.86	0.60	0.71	10
accuracy			0.52	44
macro avg	0.54	0.54	0.53	44
weighted avg	0.55	0.52	0.53	44

Extra Trees

Будую модель

```
In [51]: extra_trees_model = ExtraTreesClassifier(
          n_estimators=200,
          max_depth=8,
          random_state=42,
          class_weight='balanced',
        )
extra_trees_model.fit(x_train, y_train)
```

Out[51]:



Оцінюю точність

In [52]:

```
extra_trees_pred = evaluate_classifier('Extra Trees', extra_trees_model)
pd.Series(classification_results, name='Accuracy').sort_values(ascending=False)
```

Extra Trees accuracy = 0.5455

	precision	recall	f1-score	support
East Asia & Pacific	0.29	0.29	0.29	7
Europe & Central Asia	0.69	0.75	0.72	12
Latin America & Caribbean	0.29	0.25	0.27	8
Middle East & North Africa	0.50	0.25	0.33	4
North America	1.00	1.00	1.00	1
South Asia	0.00	0.00	0.00	2
Sub-Saharan Africa	0.69	0.90	0.78	10
accuracy			0.55	44
macro avg	0.49	0.49	0.48	44
weighted avg	0.51	0.55	0.52	44

Out[52]:

	Accuracy
Extra Trees	0.545455
Random Forest	0.522727
Decision Tree	0.477273
KNN	0.454545
SVC	0.454545

Висновок

Порівнюючи всі моделі, роблю висновок, що **модель Extra Trees є найкращою** для цього розбиття даних: вона має найбільшу асигасу на тестовій вибірці.

Завдання #5:

Для однієї з моделей виконайте підбір параметра. Обґрунтуйте ваш вибір.

Виконую підбір параметра `max_depth` для моделі `ExtraTreesClassifier`.

Для цього ініціалізую алгоритм з `max_depth` від 1 до 20 і для кожної моделі рахую mean accuracy на тестовій вибірці:

```
In [53]: depth_values = range(1, 21)
extra_trees_depth_scores = []

for depth in depth_values:
    model = ExtraTreesClassifier(
        n_estimators=200,
        max_depth=depth,
        random_state=42,
        class_weight='balanced',
    )
    model.fit(x_train, y_train)
    extra_trees_depth_scores.append(model.score(x_test, y_test))

best_depth_index = int(np.argmax(extra_trees_depth_scores))
best_depth = list(depth_values)[best_depth_index]
best_depth_score = extra_trees_depth_scores[best_depth_index]

print(f'Best max_depth = {best_depth}')
print(f'Best accuracy = {best_depth_score:.4f}')

pd.DataFrame({
    'max_depth': list(depth_values),
    'accuracy': extra_trees_depth_scores,
}).round(4)
```

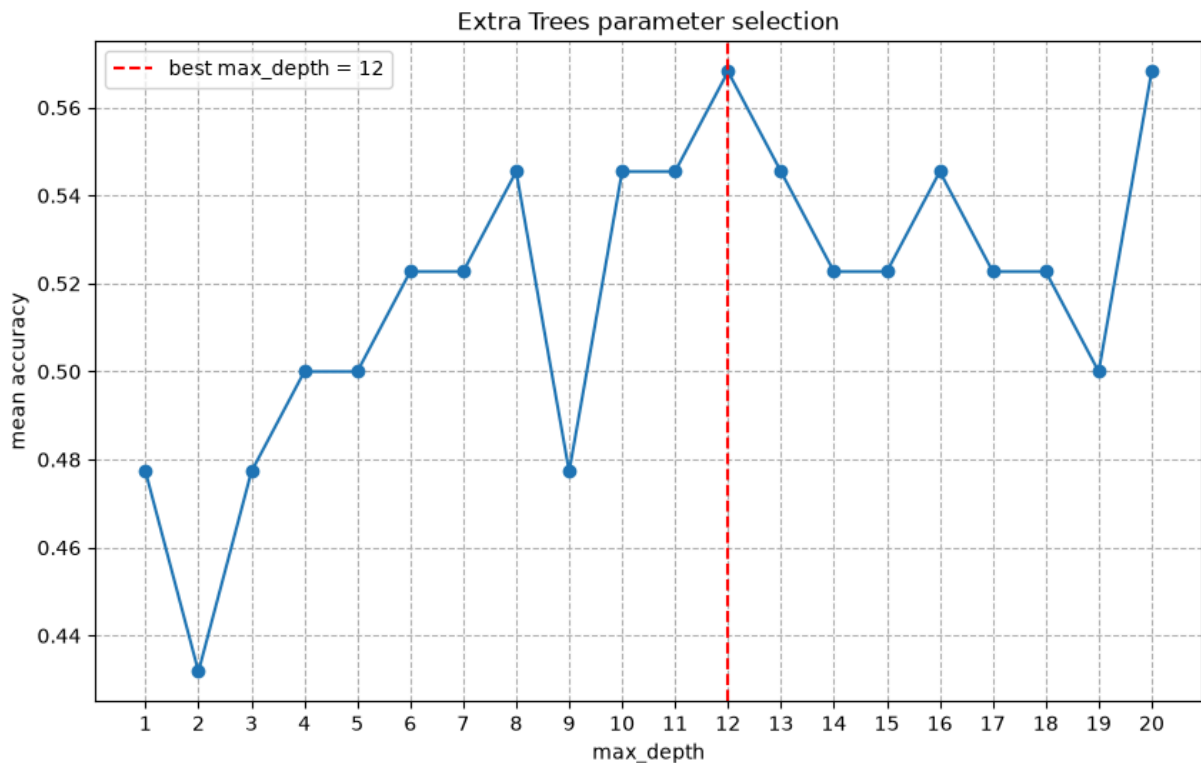
```
Best max_depth = 12
Best accuracy = 0.5682
```

Out [53]:

	max_depth	accuracy
0	1	0.4773
1	2	0.4318
2	3	0.4773
3	4	0.5000
4	5	0.5000
5	6	0.5227
6	7	0.5227
7	8	0.5455
8	9	0.4773
9	10	0.5455
10	11	0.5455
11	12	0.5682
12	13	0.5455
13	14	0.5227
14	15	0.5227
15	16	0.5455
16	17	0.5227
17	18	0.5227
18	19	0.5000
19	20	0.5682

Отримані показники якості візуалізую на графіку:

```
In [54]: plt.figure(figsize=(10, 6))
plt.plot(list(depth_values), extra_trees_depth_scores, marker='o')
plt.axvline(best_depth, color='red', linestyle='--', label=f'best max_depth')
plt.xticks(list(depth_values))
plt.xlabel('max_depth')
plt.ylabel('mean accuracy')
plt.title('Extra Trees parameter selection')
plt.grid(linestyle='--')
plt.legend()
plt.show()
```



З графіку бачу, що найкраще значення параметра `max_depth` для цього розбиття починається з 12: асигасу досягає максимуму, а подальше збільшення глибини не дає стабільного покращення. Тому обираю `max_depth = 12`.

Додаткове завдання:

Використовуючи файл [Data5.csv](#):

1. Визначити кластер країн з найкращим розвитком (кластеризувати по `le`, `lec`, `ls`; для `k`-середніх використати 4 кластера). Побудувати центри кластерів.
2. Провести кластеризацію по `CqI`, порахувати скільки країн потрапило в різні кластери, якщо порівнювати з `p1`.

Згідно з методологією вимірювання сталого розвитку країн, сталий розвиток оцінюється за допомогою відповідного індексу у просторі трьох вимірів: економічного (`lec`), екологічного (`le`) і соціально-інституціонального (`ls`). Цей індекс є вектором, норма якого визначає рівень сталого розвитку, а його просторове положення в системі координат (`lec, le, ls`) характеризує міру «гармонійності» цього розвитку.

► Натисніть [тут](#), щоб побачити підказку до першого завдання

```

In [55]: development_df = pd.read_csv(
        'Data5.csv', encoding='windows-1251', sep=';', decimal=',',
    ).rename(columns={'Unnamed: 0': 'Country'})

development_features = ['Ie', 'Iec', 'Is']
development_scaler = StandardScaler()
development_scaled = development_scaler.fit_transform(development_df[development_features])

development_kmeans = KMeans(n_clusters=4, **kmeans_kwargs)
development_kmeans.fit(development_scaled)

development_df['Development cluster'] = development_kmeans.labels_

development_centers = pd.DataFrame(
    development_scaler.inverse_transform(development_kmeans.cluster_centers_
    columns=development_features,
)
development_centers['Mean index'] = development_centers[development_features].mean(axis=1)
development_centers.index.name = 'Cluster'

best_development_cluster = int(development_centers['Mean index'].idxmax())
print(f'Cluster with the best development: {best_development_cluster}')
print(f'Number of countries in this cluster: {(development_df["Development cluster"] == best_development_cluster).sum()}')

display(development_centers.round(4).sort_values('Mean index', ascending=False))
development_df.loc[
    development_df['Development cluster'] == best_development_cluster,
    ['Country', 'ISO', 'Cql', 'Ie', 'Iec', 'Is'],
].sort_values('Cql', ascending=False)

```

Cluster with the best development: 2
 Number of countries in this cluster: 26

	Ie	Iec	Is	Mean index
Cluster				
2	0.7677	0.6835	0.6471	0.6994
0	0.6483	0.5552	0.5579	0.5871
3	0.5688	0.4280	0.4754	0.4907
1	0.2793	0.3944	0.3990	0.3576

Out [55]:

	Country	ISO	Cql	le	lec	Is
5	Australia	AUS	1.457611	0.791517	0.721155	0.692414
120	Switzerland	CHE	1.454859	0.771155	0.770265	0.669772
125	United Kingdom	GBR	1.443573	0.809062	0.698265	0.679464
35	Denmark	DNK	1.441151	0.803673	0.679001	0.698137
47	Germany	DEU	1.436559	0.748135	0.767174	0.664004
112	Singapore	SGP	1.429509	0.792138	0.781236	0.624389
91	New Zealand	NZL	1.427049	0.812911	0.701622	0.654466
126	United States	USA	1.426584	0.758169	0.774111	0.643449
63	Japan	JPN	1.423174	0.718327	0.785976	0.656615
23	Canada	CAN	1.405515	0.764216	0.737761	0.637167
43	Finland	FIN	1.397621	0.822361	0.634745	0.671395
95	Norway	NOR	1.394098	0.793581	0.639835	0.682434
6	Austria	AUT	1.393557	0.771155	0.640078	0.698254
119	Sweden	SWE	1.379451	0.813006	0.655700	0.639118
90	Netherlands	NLD	1.370201	0.738193	0.689534	0.646744
59	Ireland	IRL	1.366731	0.790791	0.664097	0.632839
55	Iceland	ISL	1.354995	0.819965	0.630008	0.631824
74	Luxembourg	LUX	1.332350	0.778058	0.641951	0.624375
40	Estonia	EST	1.326366	0.798270	0.661003	0.594485
26	Chile	CHL	1.325461	0.683625	0.676489	0.640437
116	Spain	ESP	1.321427	0.803177	0.613327	0.623657
44	France	FRA	1.309040	0.794505	0.593240	0.634039
34	Czech Republic	CZE	1.305630	0.749434	0.645762	0.612366
67	Korea, Republic of	KOR	1.289813	0.577603	0.713234	0.629807
11	Belgium	BEL	1.287539	0.696510	0.610619	0.652020
73	Lithuania	LTU	1.284121	0.761491	0.643759	0.589677

► Натисніть тут, щоб побачити підказку до другого завдання

```
In [56]: cql_scaler = StandardScaler()
cql_scaled = cql_scaler.fit_transform(development_df[['Cql']])

cql_kmeans = KMeans(n_clusters=4, **kmeans_kwargs)
cql_kmeans.fit(cql_scaled)
```

```

development_df['Cql cluster'] = cql_kmeans.labels_

cql_centers = pd.DataFrame(
    cql_scaler.inverse_transform(cql_kmeans.cluster_centers_),
    columns=['Cql center'],
)
cql_centers.index.name = 'Cql cluster'

comparison_matrix = pd.crosstab(
    development_df['Development cluster'],
    development_df['Cql cluster'],
)

# Порівнюю номери кластерів через середнє Cql у кластерах розвитку,
# бо KMeans може присвоїти однаковим за змістом групам різні номери.
development_cluster_cql_centers = development_df.groupby('Development cluster')['Cql center'].mean()
mapped_cql_clusters = cql_kmeans.predict(cql_scaler.transform(development_df[['Cql center']].values))
cluster_mapping = {
    int(development_cluster): int(cql_cluster)
    for development_cluster, cql_cluster in zip(development_cluster_cql_centers.index, mapped_cql_clusters)
}

development_df['Mapped Cql cluster'] = development_df['Development cluster'].map(cluster_mapping)
development_df['Same cluster group'] = development_df['Mapped Cql cluster']

different_clusters_count = int((~development_df['Same cluster group']).sum())
matching_clusters_count = int(development_df['Same cluster group'].sum())

print('Centers of Cql clusters:')
display(cql_centers.round(4).sort_values('Cql center', ascending=False))

print('Development clusters vs Cql clusters:')
display(comparison_matrix)

print(f'Development cluster to Cql cluster mapping: {cluster_mapping}')
print(f'Countries in matching clusters: {matching_clusters_count}')
print(f'Countries in different clusters: {different_clusters_count}')

```

Centers of Cql clusters:

Cql center	
Cql cluster	
1	1.3616
3	1.0655
0	0.7880
2	0.4738

Development clusters vs Cql clusters:

Cql cluster	0	1	2	3
Development cluster				
0	0	3	0	31
1	3	0	27	0
2	0	26	0	0
3	37	0	1	4

Development cluster to Cql cluster mapping: {0: 3, 1: 2, 2: 1, 3: 0}
 Countries in matching clusters: 121
 Countries in different clusters: 11

Виконав студент групи ІП-45
Янов Богдан